

Telegram-бот «Личные границы» — техническое решение

На основе общей концептуальной схемы процессов. LLM через Openrouter API.

Раздел 1. Части процесса и решения

«Вход / Выбор режима» — aiogram 3.x + встроенный FSM

Бот принимает /start, показывает меню; выбор режима фиксируется в FSM и запускает соответствующий процесс (помнит на каком шаге находится каждый пользователь).

«Описать ситуацию (текст/голос)» — aiogram 3.x: MessageHandler (text / voice)

Хэндлер принимает текст или голосовое сообщение; голос скачивается как .ogg и передаётся на транскрипцию.

«Описать ситуацию (голос) → AI-анализ сообщения» — Vosk (русская модель, локально, бесплатно)

Голосовой .ogg распознаётся локально моделью Vosk; не требует API-ключей, работает на слабых машинах, поддерживает русский язык.

«Сохранить сообщение в историю» — SQLite (старт) | PostgreSQL (масштаб)

Каждое сообщение (user_id, роль, текст, timestamp) INSERT-ом пишется в таблицу messages; SQLite не требует отдельного сервера.

«RAG-поиск: база знаний + диалоги других» — ChromaDB + sentence-transformers:

paraphrase-multilingual-MiniLM-L12-v2

Текст сообщения векторизуется многоязычной моделью (поддерживает русский); ChromaDB возвращает top-k релевантных фрагментов из базы знаний и диалогов других пользователей.

«Применить методологию и промт» — YAML-файлы промтов + Python template

Системный промт хранится в .yaml; перед запросом к LLM в него подставляются история, сводка и RAG-результаты.

«История диалога пользователя» — SQLite — таблица messages (SELECT последних N строк)

При сборке контекста запрос возвращает последние 10–20 сообщений пользователя; лимит по токенам контролируется обрезкой.

«RAG-поиск + история + сводка → AI-анализ сообщения + контекст» — Gemini 2.0 Flash через Openrouter API (openai>=1.0.0 совместимый клиент)

Собранный контекст отправляется в Gemini Flash через Openrouter; используется OPENROUTER_API_KEY, клиент совместим со стандартным openai SDK.

«Сформировать ответ по методологии» — Gemini 2.0 Flash через Openrouter с system prompt

Модель получает полный контекст и возвращает структурированный ответ; вызов идёт через Openrouter с ключом OPENROUTER_API_KEY.

«Получить ответ бота» — aiogram: bot.send_message()

Готовый текст отправляется пользователю; при превышении лимита Telegram (4096 символов) ответ автоматически разбивается на части.

«Ежедневная сводка по пользователю» — SQLite — таблица summaries (поле summary в user_profiles)

Сводка хранится как текст и читается при каждой новой сборке контекста; обновляется раз в сутки планировщиком.

«Запуск ежедневной обработки профилей (Планировщик)» — APScheduler (Python) | системный cron

Задача срабатывает раз в сутки, выбирает из БД пользователей с диалогами за день и запускает формирование сводки для каждого.

«AI-формирование сводки пользователя» — Gemini 2.0 Flash через Openrouter (краткий промт)

Диалоги за день сжимаются в 3–5 предложений; вызов через Openrouter с тем же OPENROUTER_API_KEY.

«Сохранить сводку в профиль» — SQLite — UPDATE user_profiles SET summary = ... WHERE user_id = ...

Старая сводка перезаписывается; при необходимости аудита архивируется в таблицу summaries_history.

«Нарезка документов для базы знаний» — langchain: только пакет langchain-text-splitters

(RecursiveCharacterTextSplitter)

При загрузке админом PDF/TXT документ разрезается на смысловые куски (~500–1000 символов с перекрытием), каждый кусок векторизуется и складывается в ChromaDB.

«Просмотр диалогов / Оценка качества / Изменение методологии / Загрузка документов (Админ-панель)» — Streamlit (Python)

Streamlit позволяет собрать полноценный веб-интерфейс без фронтенд-разработки: таблицы диалогов, форма редактирования промтов, загрузка файлов — всё в одном .py.

«Хранилище: Диалоги / Профили / Сводки» — SQLite (старт) | PostgreSQL (масштаб)

Три сущности в одной БД: таблицы messages, user_profiles, summaries_history; SQLite достаточно до ~10 000 пользователей.

«Хранилище: База знаний (векторы)» — ChromaDB (персистентное локальное хранилище)

Клиент ChromaDB сохраняет векторы на диск в папке data/chroma/; не требует внешнего сервиса и работает полностью локально.

«Хостинг» — Локальный запуск (свой ПК) или собственный VPS

Весь проект (бот + Vosk + ChromaDB + SQLite + Streamlit) запускается на одной машине в одном процессе; для простоты все компоненты интегрированы в один скрипт.

Примечания по реализации

Openrouter API: единая точка доступа к Gemini 2.0 Flash и другим LLM. Используется openai>=1.0.0 совместимый клиент — не требуется отдельный google-generativeai пакет. Ключ OPENROUTER_API_KEY прописывается в .env.

Vosk модель: путь к локальной русской модели Vosk указывается в VOSK_MODEL_PATH в файле .env; модель скачивается один раз и хранится локально.

Streamlit используется для админ-панели и входит в зависимости requirements.txt.

Раздел 2. Структура папок и файлов проекта

Ниже показана реальная файловая структура проекта. Каждый файл сопровождается назначением.

```
transformation-bot/
├── bot/ – логика Telegram-бота
│   ├── main.py – точка входа: инициализация бота, регистрация хэндлеров, запуск polling
│   ├── states.py – FSM-состояния: шаги диалога (выбор режима, ожидание ввода и т.д.)
│   └── handlers/ – хэндлеры по режимам
│       ├── onboarding.py – первый вход: приветствие, сбор имени, создание профиля
│       ├── situation.py – режим «Ситуация»: приём текста/голоса, вызов AI, отправка ответа
│       └── trainer.py – режим «Тренажёр»: генерация ситуаций, оценка ответов пользователя
├── ai/ – вся AI-логика
│   ├── llm_client.py – обёртка над Openrouter API (Gemini 2.0 Flash): send_message(), использует OPENROUTER_API_KEY
│   ├── rag.py – RAG: векторизация запроса, поиск в ChromaDB, возврат top-k фрагментов
│   ├── transcription.py – транскрипция голоса: скачать .ogg, обработать через Vosk, вернуть текст
│   └── summarizer.py – ежедневная сводка: промт + вызов LLM + сохранение результата
├── prompts/ – все системные промты
│   ├── situation_prompt.yaml – промт для режима «Ситуация»: методология анализа личных границ
│   ├── trainer_prompt.yaml – промт для режима «Тренажёр»: генерация и оценка ситуаций
│   └── summary_prompt.yaml – промт для ежедневной сводки: краткая инструкция по анализу диалогов
├── db/ – работа с базой данных
│   ├── database.py – инициализация SQLite/PostgreSQL, создание таблиц при первом запуске
│   ├── models.py – схема таблиц: messages, user_profiles, summaries_history
│   └── queries.py – функции: save_message(), get_history(), update_summary() и др.
└── knowledge/ – управление базой знаний
```

- | └─ indexer.py – загрузка документов, нарезка на чанки (langchain-text-splitters), индексирование в ChromaDB
- | └─ scheduler/ – автоматические задачи
- | └─ daily_job.py – APScheduler: запуск раз в сутки, обход активных пользователей, вызов summarizer
- | └─ admin/ – Streamlit-панель администратора
- | └─ app.py – главный файл Streamlit: навигация между страницами
- | └─ pages/ – страницы панели
- | └─ dialogs.py – таблица диалогов пользователей, фильтрация, оценка ответов бота
- | └─ methodology.py – просмотр и редактирование YAML-промтов
- | └─ knowledge.py – загрузка документов: file uploader → вызов knowledge/indexer.py
- | └─ data/ – данные (создаётся автоматически, не в git)
- | └─ bot.db – файл SQLite: диалоги, профили, сводки
- | └─ chroma/ – персистентное хранилище векторов ChromaDB
- | └─ config.py – глобальные настройки: пути, лимиты истории, модель LLM, chunk_size
- | └─ requirements.txt – зависимости: aiogram, langchain-text-splitters, chromadb, vosk, streamlit, apscheduler, openai>=1.0.0
- | └─ .env – переменные окружения: TELEGRAM_TOKEN, OPENROUTER_API_KEY, DB_PATH, VOSK_MODEL_PATH (не в git)